

Implementación y *Mantenimiento* del producto final de software

El software no termina cuando compila: se entrega, se pone en producción y vive años evolucionando. Aquí está el mapa completo de esa última — y más larga — etapa del ciclo de vida.

Miguel Figuera
Autor

Profa. Misley Baute
Docente de la unidad curricular

Julio de 2026
La Victoria, edo. Aragua

01 DE LA CONSTRUCCIÓN A LA OPERACIÓN ¿Qué es la implementación (implantación)?

Es la fase en la que el sistema, ya construido y verificado, se **instala en su ambiente real de operación** y se pone a disposición de los usuarios finales. Marca la transición del proyecto: el producto deja de ser un artefacto del equipo de desarrollo y pasa a ser una herramienta de trabajo de la organización.

Una implantación mal planificada puede hacer fracasar un sistema técnicamente perfecto: la resistencia al cambio, los datos mal migrados y la falta de capacitación destruyen la confianza del usuario en el primer contacto.

**Instalación y despliegue**
Preparar servidores, dispositivos y configuración del ambiente de producción.

**Migración de datos**
Convertir y cargar los datos del sistema anterior, verificando su integridad.

**Capacitación**
Entrenar a los usuarios finales y al personal de soporte en el nuevo sistema.

**Documentación**
Manuales de usuario y de operación; guías de instalación y respaldo.

**Aceptación**
Pruebas de aceptación con el cliente y firma de conformidad de la entrega.

02 ¿CÓMO SE APAGA EL SISTEMA VIEJO? Estrategias de conversión al nuevo sistema

RIESGO ALTO · COSTO BAJO
Conversión directa (big-bang)
ANTERIOR
NUEVO
RIESGO ●●●
El sistema anterior se apaga y el nuevo arranca de inmediato. Rápida y económica, pero sin red de seguridad: si el nuevo falla, la operación se detiene.

RIESGO BAJO · COSTO ALTO
Conversión en paralelo
ANTERIOR
NUEVO
RIESGO ●○○
Ambos sistemas operan a la vez durante un período y se comparan resultados. Máxima seguridad, pero duplica el trabajo y el costo de operación.

RIESGO CONTROLADO · APRENDIZAJE
Conversión piloto
ANTERIOR
PILOTO
RIESGO ●●○
El nuevo sistema se prueba primero en un área o sede limitada; validado el piloto, se extiende al resto de la organización con las lecciones aprendidas.

RIESGO MEDIO · GRADUAL
Conversión por fases
ANTERIOR
NUEVO
RIESGO ●●○
El sistema se implanta módulo por módulo (p. ej., primero citas, luego pagos). Reparte el riesgo en el tiempo a cambio de convivir con dos sistemas.

03 · EL DATO QUE CAMBIA LA PERSPECTIVA

El mantenimiento domina el costo del ciclo de vida



Diversos estudios estiman que entre el 60 % y el 80 % del costo total de un sistema se consume después de su entrega (Sommerville, 2011; Pressman, 2010). Diseñar pensando en la mantenibilidad — código limpio, pruebas automatizadas, documentación de decisiones — es la inversión más rentable de todo el proyecto.

04 ISO/IEC 14764 · NORMA INTERNACIONAL DEL MANTENIMIENTO Los cuatro tipos de mantenimiento

≈ 21 %
Correctivo

Repara defectos detectados en operación: fallas de lógica, errores de cálculo, caídas del sistema.
EJEMPLO · Un pago registrado no aparece en el historial del cliente.

≈ 25 %
Adaptativo

Ajusta el software a cambios del entorno: nuevo sistema operativo, nueva versión del API, nuevas leyes.
EJEMPLO · Migrar la app a una nueva versión de Android o del SDK.

≈ 50 %
Perfectivo

Mejora funciones existentes o añade nuevas a petición del usuario; incluye mejoras de rendimiento.
EJEMPLO · Agregar recordatorios automáticos de vacunación.

≈ 4 %
Preventivo

Reestructura y refactoriza para facilitar el mantenimiento futuro antes de que los defectos aparezcan.
EJEMPLO · Refactorizar listas a componentes virtualizados.

Distribución típica del esfuerzo según los estudios clásicos de Lientz y Swanson, citados por Pressman (2010).

05 NADA SE TOCA SIN CONTROL DE CAMBIOS El proceso de mantenimiento, paso a paso

1
Solicitud de cambio
Usuarios o soporte reportan defectos o piden mejoras (tickets, incidencias).

→

2
Análisis de impacto
Se evalúa qué módulos, datos y usuarios afecta; se estima costo y riesgo.

→

3
Priorización
El comité de cambios aprueba, pospone o rechaza; lo urgente entra como corrección.

→

4
Implementación
Se modifica el código con control de versiones y revisión por pares.

→

5
Pruebas de regresión
Se verifica el cambio y que nada de lo que funcionaba se haya roto.

→

6
Liberación
Nueva versión numerada, notas de cambio y despliegue a los usuarios.

06 POR QUÉ EL SOFTWARE NUNCA ESTÁ “TERMINADO” Leyes de la evolución del software (Lehman)

I Cambio continuo
Un sistema que se usa en el mundo real debe adaptarse continuamente o se vuelve progresivamente menos útil.

II Complejidad creciente
Al evolucionar, la estructura del sistema se degrada y su complejidad aumenta, salvo que se trabaje activamente para reducirla.

VI Crecimiento continuo
La funcionalidad debe crecer a lo largo de la vida del sistema para mantener satisfechos a sus usuarios.

VII Calidad decreciente
La calidad percibida decae si el sistema no se adapta con rigor a los cambios de su entorno operativo.

CASO DE ESTUDIO REAL · PROYECTO DE LA UNIDAD CURRICULAR

Cómo lo aplica Pawcare

La aplicación móvil de atención veterinaria a domicilio del consultorio Pawcare (Expo SDK 56 · React Native · TypeScript, sobre un backend API REST de 157 rutas) ya practica la implantación y el mantenimiento disciplinados:

**Implantación por fases**
El producto se entrega en incrementos por dominio — público, dueño y administrador — validados con la cliente en cada revisión de sprint.

**Regresión real, no simulada**
15 suites con 49 pruebas de integración corren contra el backend real antes de cada commit: la red de seguridad del mantenimiento.

**Control de cambios**
Ramas cortas, pull requests con revisión y commits convencionales: cada cambio queda trazado desde la solicitud hasta la liberación.

**Compuertas automáticas**
Pruebas, código, lint y react-doctor bloquean el push de código defectuoso: mantenimiento preventivo integrado al proceso.

**Despliegue móvil**
Builds desplegados con Expo y actualizaciones controladas: liberar una corrección no exige reinstalar la aplicación completa.

**Memoria de decisiones**
Los ADR (registros de decisión arquitectónica) documentan el porqué de cada diseño, reduciendo el costo de entender el sistema en el futuro.

REFERENCIAS

ISO/IEC. (2022). ISO/IEC/IEEE 14764: Software engineering — Software life cycle processes — Maintenance. · Lehman, M. M. (1980). Programs, life cycles, and laws of software evolution. *Proceedings of the IEEE*, 68(9). · Pressman, R. (2010). *Ingeniería del software: un enfoque práctico* (7.ª ed.). McGraw-Hill. · Sommerville, I. (2011). *Ingeniería de software* (9.ª ed.). Pearson.